NOVA IMS
INFORMATION MANAGEMENT SCHOOL

Machine Learning Operations
Master in Data Science and Advanced Analytics

NOVA Information Management School
Universidade Nova de Lisboa

Taxi Tip Amount

Regression Problem

Lara Santos, 20221823

André Nicolau, 20221918

Afonso Hermenegildo, 20221958

Marco Martins, 20221951

Github Link: https://github.com/afonsosousah/MLOPS-Project

Spring Semester 2025/2026

Table of Contents

1. Introduction

1.1. Problem Overview

Tipping is an important part of the urban mobility industry but is difficult to predict because it depends on factors such as trip distance, fare amount, time of day, pickup/drop-off location, and route type. This variability complicates financial planning for drivers and revenue forecasting for fleet operators. The large volume of New York City taxi trips and the availability of detailed public data make this a suitable machine learning problem.

This project uses the official NYC Taxi and Limousine Commission (TLC) Green Taxi Trip Record Data, released monthly in Parquet format. The dataset contains millions of card-payment trips with information on fares, distances, locations, timestamps, surcharges, and tip amounts. The objective is to develop an end-to-end MLOps pipeline that predicts passenger tip amounts using only information available at the time of payment. Beyond prediction, the model supports trip simulation through a serving API, helping drivers and fleet operators estimate tip revenue, improve trip selection, and support pricing decisions. The pipeline also incorporates automated data quality checks, MLflow experiment tracking, and a scalable model suitable for production monitoring.

Green taxi data from January 2024 to April 2026 (28 monthly files) was used. Data from January 2024 - June 2025 was used for training, July 2025 for validation, and January - April 2026 for production drift evaluation. Only card-payment trips (payment_type = 1) were included because cash tips are unavailable. The field cbd_congestion_fee, introduced in 2025, was removed to ensure a consistent schema.

1.2. Success Metrics

Given the regression nature of the problem, we considered Mean Absolute Error (MAE) as the primary evaluation metric, as it is directly interpretable in dollar terms and clearly communicates prediction quality to non-technical stakeholders. We also considered Root Mean Squared Error (RMSE) to capture sensitivity to larger errors, R^2 to measure the proportion of variance explained by the model, and Median Absolute Error (MedAE), as it provides a robust measure of prediction error that is less affected by outliers.

2. Project Planning

The project was organised into three one-week agile sprints, following the Kedro pipeline components as vertical slices of deliverable work. In this way, a new functional pipeline was delivered for testing at the end of every sprint:

Sprint 1 - Data Foundations: Ingested the 28 monthly TLC Parquet files, generated a *ydata-profiling* report to understand data distributions, and developed a Great Expectations validation suite for the Kedro pipeline. A schema audit identified the cbd_congestion_fee field introduced in 2025, which was removed to maintain a consistent schema.

Sprint 2 - Preprocessing, Feature Engineering and Feature store: Cleaned the data by filtering card-payment trips, removing invalid records, and deriving eight new features. The processed dataset was stored in Hopsworks (green_taxi_features, version 2), and a time-based split was applied for training, validation, and testing: the training set contains data from January 2024 to June 2025, the validation set spans from July 2025 through the end of that year, and the test set comprises all available 2026 data up until April.

Sprint 3 - Modelling, Serving, and Monitoring layers: Ran the model selection pipeline comparing different models with an Optuna search on top, tracked all experiments and trials in MLflow, and generated SHAP explainability for the selected model. Built a FastAPI serving endpoint, deployed it in Docker, performed batch scoring on 2026 data, generated a NannyML drift report, and implemented pytest tests for the main pipeline components.

3. Data Exploration & Modeling Results

3.1. Data Exploration

Before exploration, the data was split into a train/reference set and a test/analysis set, to avoid leakage and allow us to test the expectations. The train set includes 2024-2025 trips and was used for profiling, EDA, building the expectations and cleaning decisions. The test set includes January-April 2026 trips and was reserved for expectations testing, model predictions and drift evaluation.

The train set contains 1,134,147 trips and 20 variables, with no duplicate rows. Overall missingness was 7.1% across the dataset. The ehail_fee column was fully missing and was removed. Six operational fields, RatecodeID, passenger_count, payment_type, trip_type, store_and_fwd_flag, and congestion_surcharge, had about 6.9% missing values each. trip_distance was strongly right-skewed and included 4.6% zero-distance records, which were removed during cleaning.

The target, tip_amount, was also right-skewed, with an average value of $2.61 and 36.5% of records showing a zero tip (Figure 1). Some raw values were invalid or extreme, ranging from -$100 to $460, so the cleaning step kept only tips in the range [0, 200).

The correlation matrix (Figure 2) showed that fare_amount had the strongest linear relationship with tip_amount (r = 0.354), followed by trip_distance (0.299), trip_type (0.183), and DOLocationID (0.172). Other variables, such as RatecodeID, passenger_count, and PULocationID, had weak linear correlations. Since total_amount includes the tip, it was removed to avoid target leakage. The remaining fare and distance variables were kept because they provide useful trip-level signals for the model.

3.2. Data Quality Tests

Data quality testing was implemented with Great Expectations using the green_taxi_quality_v1 validation suite. The suite was built based on the train data and contains 24 expectations covering schema, completeness, value ranges, valid code sets, and timestamp consistency.

The checks verify that the expected Green Taxi columns are present and that key fields such as pickup and dropoff timestamps, pickup and dropoff location IDs, trip distance, fare amount, and total amount are not null. Location IDs are required to fall between 1 and 265, while trip_distance, fare_amount, and total_amount are expected to be non-negative, with tolerance for rare adjustment records. The suite also checks valid code sets for fields such as RatecodeID, payment_type, trip_type, and store_and_fwd_flag, and verifies that dropoff time occurs after pickup time.

When the same suite was applied to the 2026 analysis batch, six expectations failed at warning severity. These failures were related to completeness in RatecodeID, payment_type, trip_type, store_and_fwd_flag, passenger_count, and congestion_surcharge. The missingness rate for these fields increased from about 5.9% in the reference data to about 14.3% in the 2026 batch. This result was kept as a data quality and monitoring finding rather than a blocking failure, since the project currently treats validation failures as warning-level evidence for production risk and drift analysis.

3.3. Feature Engineering and Selection

The feature engineering pipeline derived eight new attributes from the raw trip record (Table 1). Temporal features were extracted from the pickup timestamp: trip_duration_min, which measures the elapsed time of the journey in minutes; pickup_hour, capturing the hours of each day (zero to twenty-three); pickup_dayofweek, encoding the day of the week (Monday to Sunday); and pickup_month, capturing seasonal effects. Three binary flags were derived from these: is_weekend marks Saturday and Sunday trips, is_rush_hour marks weekday trips departing between seven and nine in the morning or five and seven in the evening, and is_night marks trips beginning between ten in the evening and five in the morning. A spatial feature, is_airport, was set to one when either the pickup or dropoff zone corresponds to JFK (zone 132), LaGuardia (zone 138), or Newark (zone 1), the three major airports served by green taxis.

Then feature selection was performed, where a set of columns was excluded to prevent leakage and redundancy. The total_amount column was dropped because it included the tip and would create a direct target leakage. The payment_type column was dropped because all remaining rows had the value one after the credit-card filter, making it a constant with no predictive value. Raw timestamp columns were replaced by the derived temporal features, and Administrative fields such as store_and_fwd_flag and VendorID were excluded as they carry no business-relevant signal towards tip prediction. The final feature set was registered in Hopsworks as the green_taxi_features feature group (version 2). This enables any downstream pipeline to retrieve a consistent, versioned snapshot of the processed features rather than re-running the cleaning from scratch.

3.4. Model and Experimentation

Model selection was implemented as a structured comparison of several regression models, with experiments tracked in MLflow. The model_selection pipeline evaluates four eligible model families under the same train-validation split: Linear Regression, Random Forest, Extra Trees, and Histogram-based Gradient Boosting.

A dummy regressor is also included as a non-eligible baseline to provide a simple reference point. For each model, the pipeline logs training and validation MAE, RMSE, and R2. An optional Optuna search was added to tune the eligible model families and select the best configuration based on validation MAE. Each trial is logged as a nested MLflow run, including parameters, metrics, and the selected model type. The selected configuration is then passed to the model_train pipeline, which trains the final production model and logs the deployment-ready artifacts.

Feature selection is also configurable. An RFE-based step is implemented, but it was disabled in the final configuration to keep the feature set easier to audit. Instead, the project relies on a curated exclusion strategy to remove leakage or low-value fields such as total_amount, payment_type, lpep_dropoff_datetime, store_and_fwd_flag, and VendorID.

3.5. Explainability (SHAP and Feature Importance)

Explainability was generated for the selected Random Forest model using SHAP. Since the final model is tree-based, the pipeline used SHAP's TreeExplainer on a validation sample to estimate how each feature contributes to the model's predictions. The results were saved as both a SHAP summary plot and a ranked table of mean absolute SHAP values. As a second view, the pipeline also generated the Random Forest's native feature importance.

4. Data Drift

The data_drift pipeline uses NannyML to compare the reference training data (cleaned) against the 2026 holdout batch. This step monitors whether production-like data has shifted from the distribution seen during training, which is important because distribution changes can reduce model reliability over time. After cleaning, the drift batch contained 102,217 rows, compared with 421,418 reference rows. NannyML was configured with two complementary drift methods. First, univariate drift was calculated for each raw feature using Jensen-Shannon distance, with the data grouped into monthly chunks. Second, NannyML's data reconstruction drift method was applied to the encoded feature matrix to detect broader multivariate changes across the full feature space.

The pipeline saves a detailed drift report and a summary file under data/08_reporting/. The generated report contains 144 drift checks: 138 univariate feature-month checks and 6 data reconstruction checks. The summary recorded 68 univariate alerts and 6 data reconstruction alerts. This indicates that several individual features changed in the 2026 holdout period, and that the overall encoded feature space also shifted across all monitored monthly chunks.

These results are treated as monitoring evidence rather than an automatic pipeline failure. The drift checks are currently non-blocking, but they highlight production risk and support continued monitoring, closer inspection of shifted features, and possible model retraining if future batches show the same pattern.

5. Final Pipeline Architecture

The complete flow of the pipeline, from ingestion to data drift detection, is illustrated in Figure 3 and Figure 4. The final implementation follows a modular and reproducible Machine Learning workflow, designed to support scalability and feature reuse. The pipeline begins with data ingestion, followed by data unit tests that ensure schema integrity and quality expectations are met before proceeding to downstream tasks. Subsequently, the data undergoes a data cleaning pipeline, which filters out anomalies and handles initial feature creation. These engineered features are then modularly written to a centralized feature store via the feature store pipeline, allowing integration with Hopsworks for feature management and logging. For model development, the workflow transitions into a rigorous chronological data splitting phase. First, the split data pipeline isolates the test set from the historical data. Next, the split train pipeline further partitions the training set into training and validation folds to ensure robust local evaluation.

The modeling track begins with the preprocessing train pipeline, which scales and transforms the training data, followed by a feature selection pipeline to optimize the feature space and reduce dimensionality. This feeds into the model selection pipeline, where multiple algorithms are evaluated and Optuna is utilized to find the optimal hyperparameters. This stage culminates in the model train pipeline, which fits the final selected model and generates SHAP interpretability plots to explain feature contributions. The trained model is packaged as a FastAPI application and served inside a Docker container to support deployment and production monitoring, ensuring environment consistency and reproducibility. The container includes the project source code, while model artefacts are mounted as a volume at runtime, enabling the model to be updated without rebuilding the image. The API exposes a /health endpoint for liveness checks and a /predict endpoint that accepts trip features and returns the predicted tip amount in real time.

6. Model Results

The final production model is a Random Forest Regressor selected by Optuna after 50 hyperparameter search trials. The selected configuration used n_estimators=500, max_depth=22, min_samples_leaf=6, and max_features=0.8.
On the training set, the model achieved an MAE of 1.02, RMSE of 1.86, Median AE of 0.59, and R2 of 0.647. On the held-out validation set, performance decreased to an MAE of 1.28, RMSE of 2.37, Median AE of 0.73, and R2 of 0.423. This train-validation gap suggests moderate overfitting, which is expected for a flexible ensemble model. However, the validation Median AE of 0.73 means that for at least half of the validation trips, the predicted tip was within less than one dollar of the actual tip amount.
Figure 5 shows the SHAP summary plot, which explains how features affect individual predictions in the validation sample. fare_amount again has the largest impact: higher fare

values generally push predicted tips upward, while lower fare values push predictions downward. congestion_surcharge, trip_distance, PU_borough_Queens, and extra also contribute meaningfully, although with smaller effects.
Figure 6 shows the Random Forest's native feature importance. fare_amount is by far the strongest predictor, followed by trip_distance, trip_duration_min, DOLocationID, and pickup_hour. This indicates that the model relies mainly on fare size, trip length, trip duration, destination area, and time of day.
Overall, both explainability views support the same conclusion: predicted tip amount is mostly driven by fare size and trip characteristics, with additional signal from location and surcharge-related variables.

7. Conclusion - Technology Choices, Risks & Mitigations

This project built a reproducible MLOps pipeline to predict NYC Green Taxi tip amounts. The solution uses uv for environment management, Kedro for pipeline orchestration, Great Expectations for data quality checks, MLflow for experiment tracking and model artifacts, Optuna for model tuning, SHAP for explainability, NannyML for drift monitoring, and FastAPI with Docker for model serving.

The final Random Forest model achieved a validation MAE of 1.28 and Median AE of 0.73, meaning that at least half of the validation predictions were within less than one dollar of the real tip amount. However, the validation R2 of 0.423 shows that tipping behavior is only partly explained by the available trip features, so the model should be used as decision support rather than as a guaranteed revenue predictor.

The main risks are data quality, drift, scalability, and overfitting. The 2026 batch showed higher missingness in some operational fields, and NannyML detected feature-level and multivariate drift. The train-validation gap also suggests moderate overfitting. In production, these risks would be mitigated with continued validation checks, non-blocking drift alerts, retraining rules, and a formal champion-challenger promotion process. If data volume grows, parts of the Pandas-based pipeline could be migrated to Spark.

The project is reproducible through the uv environment, Kedro pipeline structure, MLflow tracking, and automated tests. The final test suite passed with 53 tests. The feature-store step was implemented as an optional Hopsworks integration, while the final run used the local Kedro catalog fallback. Future work should define stricter retraining thresholds, strengthen API input validation, and decide whether Hopsworks should be enabled for a real production deployment.

The main project dependencies are listed in requirements.txt at the root of the repository.

8. Appendix

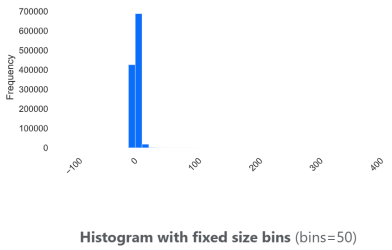

Figure 1 - Target Variable Distribution

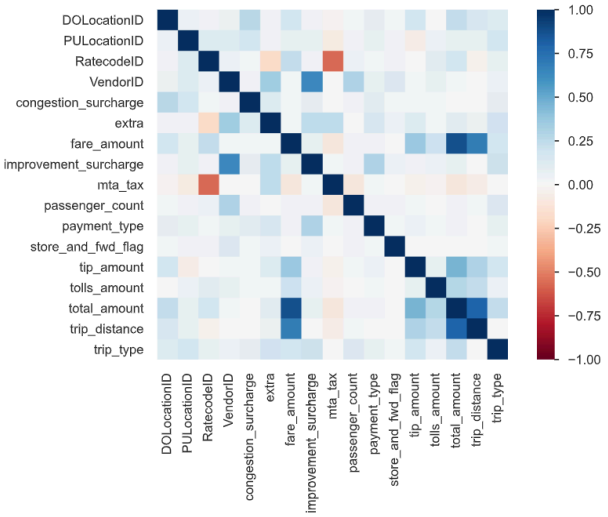

Figure 2 - Correlation Matrix

Feature	Description	Source
trip_duration_min	Trip duration in minute	pickup/dropoff datetime
pickup_hour	Hour of pickup (0–23)	lpep_pickup_datetime
pickup_dayofweek	Day of week (0=Mon, 6=Sun)	lpep_pickup_datetime
pickup_month	Month (1–12)	lpep_pickup_datetime
is_weekend	1 if Saturday or Sunday	pickup_dayofweek
is_rush_hour	1 if weekday 7–9am or 5–7pm	pickup_hour + is_weekend
is_night	1 if pickup between 10pm–5am	pickup_hour
is_airport	1 if PU or DO at JFK/LGA/EWR	PULocationID / DOLocationID

Table 1 - Feature Engineering

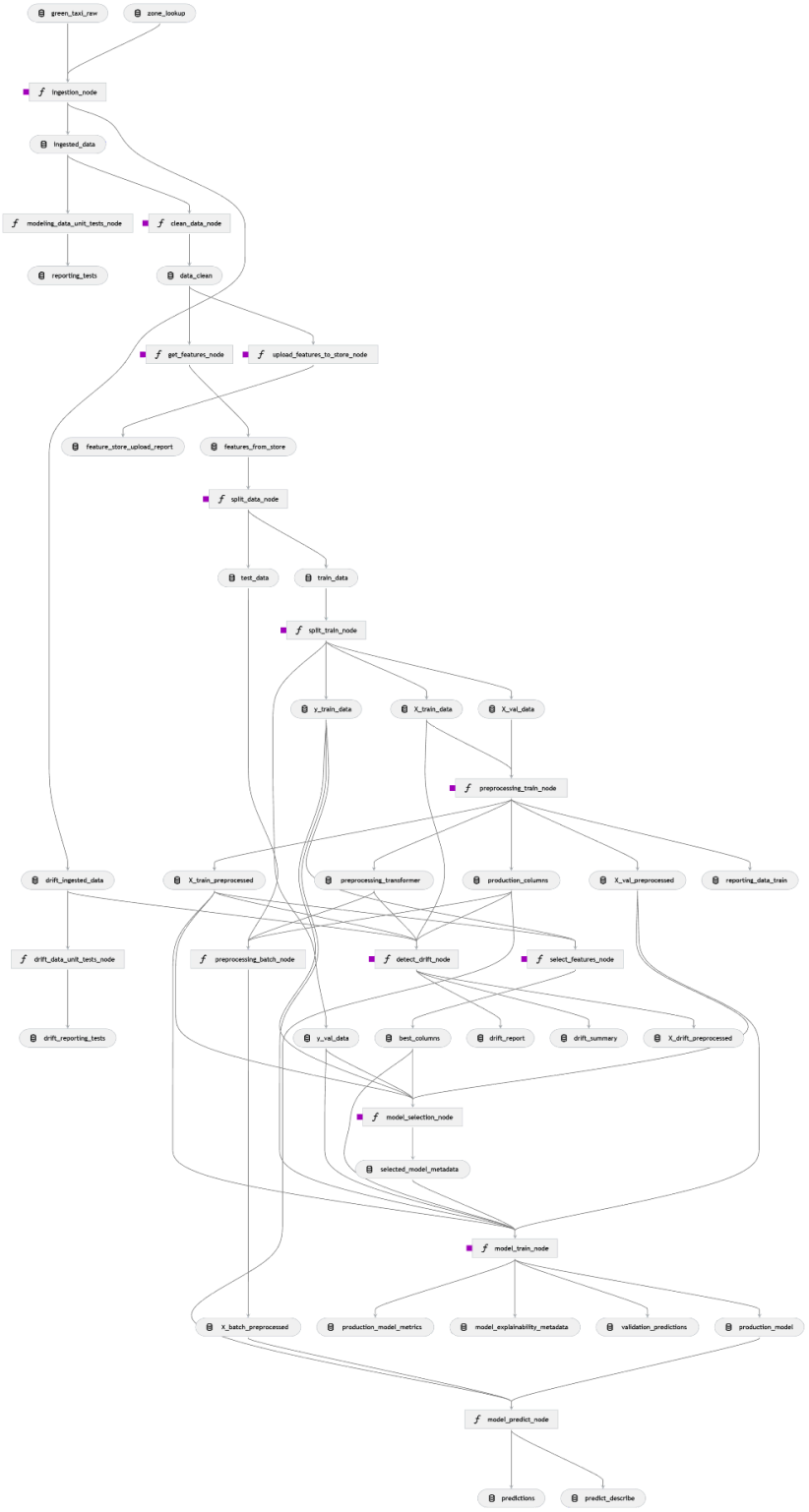

Figure 3 - Kedro Viz

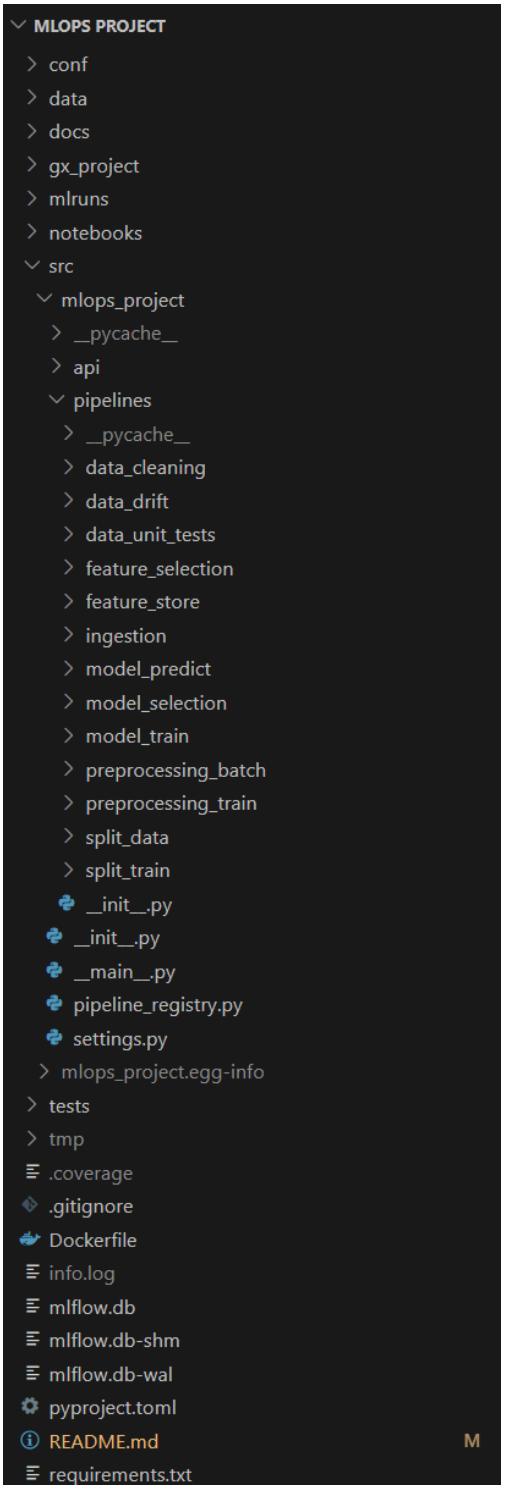

Figure 4 - Project Organization

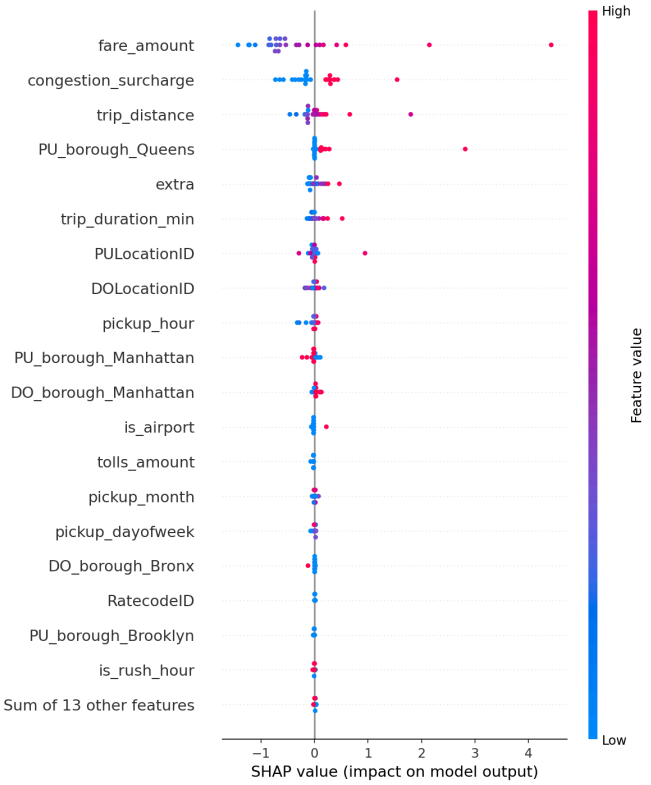

Figure 5 - SHAP Summary

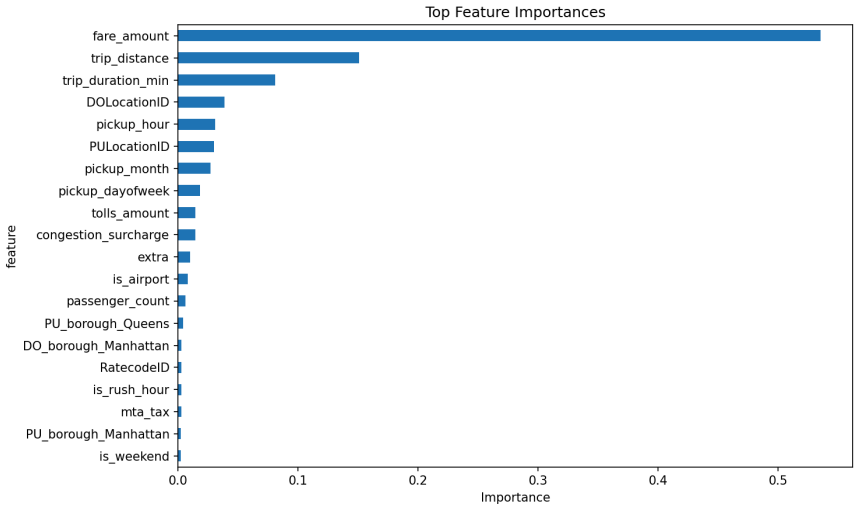

Figure 6 - Feature Importance